

심층 0.4 데이터 주도 아키텍처 리팩터링

콘텐츠를 늘릴 때 시스템 코드를 다시 고치지 않는 구조로

AI 보조 구현 → 사람의 코드 리뷰로 확장 위험 발견 → 데이터 · 행동 · 저장 · 시스템 계약 분리 → 검증

프로젝트	심층 (Shimcheung) — 세로 모바일 방치형 쿼터뷰 오토 아레나
구간	v0.3.3 → 0.4 (신규 콘텐츠 없음, 구조만 교체)
기준 커밋	0386937 (v0.3.3) → 4ef06cb (0.4)
엔진	Unity 6000.5.1f1 · C# · URP 2D
콘텐츠 버전	0.4.0-core.7 (manifest schema v3, 11개 도메인)
작성일	2026-08-12
사례 페이지	pcw0611.github.io/portfolio/case/abyss-data-driven/

이 문서의 모든 수치는 이 저장소에서 실제로 실행해 확인한 값이다. 실행하지 않은 검증은 근거로 쓰지 않고, 15장에 미검증 항목으로 따로 적었다.

1 무엇을 만드는가

한 손으로 하는 세로 화면 방치형 게임이다. 플레이어는 지하로 계속 내려가고, 전투는 자동으로 돌아간다. 결정은 **무엇을 키울 것인가**에서만 일어난다.



구조

단위	내용
층	적 24마리 · 5층마다 보스 · HP 1.25층 / 골드 1.15층
존	20층 묶음 5종. 팔레트 · 몬스터 혼합비 · BGM 루트음이 존마다 다르다
보스	7종 순환, 3종은 2페이지
적	아키타입 6종 × 엘리트 접두사 6종

성장 3축

축	재화	리셋
강화 8종	골드	회귀 시 초기화
심연 특성 12노드	심연석	영구
회귀	도달 깊이	—

이 화면에 보이는 강화 8행, 보스 이름, 존 이름, 스킬 3종은 **0.3.3까지 전부 C# 코드에 적혀 있었다**. 0.4는 게임을 하나도 바꾸지 않고 그 소유권만 옮긴 구간이다.

0.4의 계약 — v0.3.3의 전투 · 진행 · 경제 · UI · 연출을 그대로 유지한다. PlayerPrefs 키 `shimcheung.save.v1` 과 기존 JSON 필드를 유지한다. 외부 패키지 · 서버 SDK · 신규 콘텐츠는 추가하지 않는다.

2 0.3까지의 구조와, 그때는 옳았던 이유

모든 콘텐츠가 `Balance.cs` 의 정적 배열 하나에 있었다. 콘텐츠가 8행일 때 이것은 **가장 빠르고 가장 안전한 선택**이었다.

v0.3.3 — Assets/Scripts/Core/Balance.cs

```
public enum UpKind
{
    Damage = 0, Speed = 1, CritChance = 2, CritDamage = 3, Gold = 4, Companion = 5,
    Vitality = 6, Move = 7
}

public static readonly UpgradeDef[] Upgrades =
{
    new UpgradeDef {
        Kind = UpKind.Damage, Name = "검날 연마", Icon = IconId.Sword,
        BaseCost = 10, CostMul = 1.090f, Tint = new Color(1f, 0.38f, 0.34f),
        Effect = lv => "공격력 " + Num.Fmt(RawDamage(lv))
    },
    new UpgradeDef {
        Kind = UpKind.Speed, Name = "속공 수련", Icon = IconId.Speed,
        BaseCost = 32, CostMul = 1.125f, Tint = new Color(0.35f, 0.85f, 1f),
        Effect = lv => "초당 " + AttacksPerSecond(lv).ToString("0.00") + "회"
    },
    /* ... 6행 더 ... */
};

public static UpgradeDef Def(UpKind k) { return Upgrades[(int)k]; }
```

이 코드의 장점은 실제로 있었다

- 컴파일러가 오타를 잡는다. 존재하지 않는 강화를 참조하면 빌드가 깨진다.
- 파일 하나만 열면 된다. 밸런스 수정에 파이프라인이 필요 없다.
- 런타임 로딩 · 검증 코드가 0줄이다. 데이터가 곧 실행 파일이다.

0.1부터 0.3.3까지 이 구조는 비용을 청구하지 않았다. 강화 6종이 8종이 되고 보스가 7종이 될 때까지도 그랬다. 문제는 규모가 아니라 방향이었다.

0.5는 장비 6슬롯 · 4등급 · 옵션 풀을 들여오고, 그 뒤에는 원격 콘텐츠 교체와 서버 검증이 온다. 그 방향에서 이 구조는 세 가지를 동시에 요구한다 — 콘텐츠 한 행을 늘릴 때마다 **코드 수정 · 재컴파일 · 재배포**. 그리고 아래 3장의 위험은 그 시점이 아니라 **이미 지금** 코드에 들어 있었다.

3 사람이 리뷰에서 발견한 위험 4가지

0.4의 착수 이유는 성능도 버그도 아니었다. AI가 빠르게 구현한 코드를 사람이 장기 콘텐츠 규모와 인계 상황에서 다시 읽은 결과였다.

1 콘텐츠 ID와 ordinal 배열이 코드에 병렬로 고정된다

행 추가가 코드 수정으로 번지고, enum 순서가 그대로 세이브 포맷이 된다. → 4장

2 enum / switch가 콘텐츠 종류와 실행 행동을 동시에 대표한다

"무엇인가"와 "무엇을 하는가"가 한 식별자에 묶여 확장 경계가 사라진다. → 5장

3 서로 종속된 규칙이 여러 파일에 흩어진다

4 (슬롯 수), 3 (마지막 슬롯), -1 (빈 슬롯)이 저장 · 로직 · UI에 각각 적혀 있어 부분 수정에 취약하다. → 6장

4 전역 접근과 정적 데이터가 테스트 · 서버 경계 · 교체를 막는다

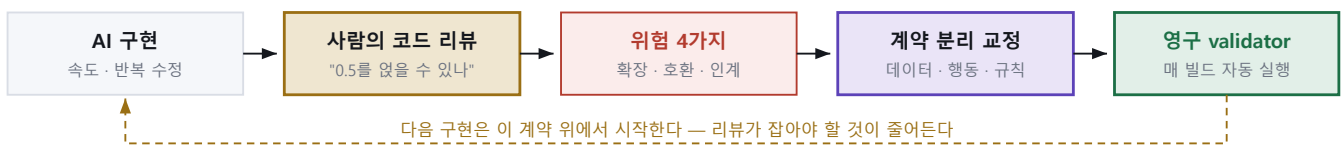
Game.I, static SaveIO 는 대역으로 바꿔 끼울 수 없다. → 7장

이 문서가 말하려는 것

이것은 **AI의 실패담이 아니다**. 네 가지 모두 동작하는 코드였고, 0.3.3은 그 상태로 실제 릴리스되어 Windows · WebGL로 배포됐다. 위험은 "지금 틀렸다"가 아니라 "**다음 버전에서 비용이 폭발한다**"는 종류였고, 그 판단에는 제품 로드맵과 인계 계획이 필요했다.

역할은 이렇게 갈렸다.

담당	말은 것
AI	구현과 반복 수정의 속도. 11개 도메인 스키마, importer, validator, 마이그레이션을 짧은 시간에 만든다
사람	제품 맥락 · 확장성 · 호환성 · 인계 위험의 최종 판단. "이 구조로 0.5를 얻을 수 있는가"



검증이 사람의 판단을 대체하지는 않는다. 대신 **한 번 내린 판단이 다음 사람에게도 강제되게** 만든다 — 계약이 문서에만 있으면 지켜지지 않고, 테스트에 있으면 빌드가 지킨다.

도식 5 — 리뷰 → 교정 → 검증 루프. AI가 속도를, 사람이 방향을 맡고, validator가 그 판단을 고정한다.

같은 원칙의 반대편 — 불필요한 추상화로 파일 추적 비용을 늘리지 않는다. 단순 초기값, 루프 증감, 수학식의 자연스러운 상수는 그대로 둔다. 분리 대상은 **도메인 의미 · 개수 · 한계 · 인덱스 관계**를 가진 값뿐이다.

4 콘텐츠 ID와 ordinal 배열의 병렬 고정

enum의 선언 순서가 저장 파일의 배열 인덱스였다. 콘텐츠 목록을 재정렬하는 순간 **모든 플레이어의 세이브가 뒤섞인다.**

V0.3.3 — SAVEDATA.CS

```
// upgrades (indexed by UpKind)
public int[] levels = new int[8];

// abyss tree (indexed by Balance.N.*)
public int[] abyssNodes = new int[Balance.N.Count];

public int Lv(UpKind k) { return levels[(int)k]; }
public void SetLv(UpKind k, int v) { levels[(int)k] = v; }
```

0.4 — SAVEDATA.CS

```
// v0.3 rollback mirror, kept in sync on save
public int[] levels;

/// Canonical upgrade levels from v7.
public StableIntValue[] upgradeLevelsById;

public int Lv(string stableId)
{
    return StableValue(upgradeLevelsById, stableId);
}
```

무엇이 실제로 위험했나

- 강화를 **중간에 하나 끼워 넣으면** 그 뒤 모든 인덱스가 한 칸씩 밀린다. 기존 세이브의 "탐욕의 손 Lv.23"이 "그림자 잔영 Lv.23"으로 읽힌다.
- enum · 표시 배열 · 세이브 배열 · UI 아이콘 배열이 **서로 다른 파일에서 같은 순서를 암묵적으로 약속**하고 있었다. 컴파일러는 이 약속을 검사하지 않는다.
- 서버가 콘텐츠를 내려주는 미래에는 **순서 자체를 클라이언트가 통제할 수 없다.**

교체 후

정본은 "upgrade.damage" 같은 **안정 문자열 ID**다. 콘텐츠 행의 순서를 바꾸거나 중간에 끼워 넣어도 세이브는 영향을 받지 않는다. ordinal은 사라지지 않고 **롤백 미러**로 남아 저장 직전에 정본에서 다시 생성된다 (12장).

228 → 0

UpKind · SkillKind · IconId ·
BossKind 직접 참조

8 / 3 / 12

안정 ID로 옮긴 강화 · 스킬 · 심연 노트

v6 → v7

세이브 버전, 기존 키 · JSON 외형 유지

5 enum이 종류와 실행을 동시에 대표했다

보스를 하나 추가하려면 enum 한 줄, 이름 배열 한 칸, switch 세 곳을 동시에 고쳐야 했다. 하나라도 빠뜨리면 조용히 기본값이 나온다.

v0.3.3 — BossCombat.cs (발췌)

```
public enum BossKind { None = -1, MineLord = 0, CrystalQueen = 1, /* ... */ SkyCalamityDragon = 6 }

static readonly string[] BossCombatNames =
{ "깡도의 주인", "수정 여왕", "용광로 폭군", "심연의 대사제", "가라앉은 왕", "균열 포식자", "천공의 재앙룡" };

public static bool BossHasSecondPhase(BossKind kind)
{
    return kind == BossKind.CrystalQueen || kind == BossKind.HighPriest ||
           kind == BossKind.SkyCalamityDragon;
}

public static Color BossTint(BossKind kind)
{
    switch (kind)
    {
        case BossKind.MineLord: return new Color(0.78f, 0.82f, 0.88f);
        case BossKind.CrystalQueen: return new Color(0.62f, 0.55f, 1.00f);
        /* ... 5 more ... */ default: return new Color(1.00f, 0.36f, 0.36f);
    }
}

public static float BossVisualHeight(BossKind kind) // 또 하나의 switch
{
    switch (kind)
    {
        case BossKind.RiftDevourer: return 350f; case BossKind.HighPriest: return 440f;
        case BossKind.SkyCalamityDragon: return 500f; default: return 420f;
    }
}
```

0.4 — ContentSource/GoogleSheets/bosses.csv (전문)

```
id,behavior_id,visual_id,name_key,ui_accent_color,visual_height,has_second_phase
boss.mine_lord,boss.behavior.mine_lord,boss.visual.mine_lord,boss.mine_lord.name,#C7D1E0,420,FALSE
boss.crystal_queen,boss.behavior.crystal_queen,boss.visual.crystal_queen,boss.crystal_queen.name,#9E8CFF,420,TRUE
boss.high_priest,boss.behavior.high_priest,boss.visual.high_priest,boss.high_priest.name,#FF59B8,440,TRUE
... 4 rows more ...
```

0.4 — BossCombat.cs 전체가 30줄로 줄었다

```
public static BossDef BossForFloor(IGameContentCatalog content, int floor)
{
    int encounter = Mathf.Max(0, floor / BossEvery - 1);
    return content.Bosses[encounter % content.Bosses.Length];
}
```

핵심 분리 — behavior_id 는 실행 가능한 엔진 계약을 가리키고, visual_id 는 렌더러 계약을 가리킨다. 둘은 독립이다. 새 보스가 기존 패턴을 재사용하면 코드는 0줄이고, 완전히 새로운 패턴이 필요하면 그때만 behavior registry에 구현을 등록한다. 어느 쪽인지가 데이터에 드러난다.

6 종속된 규칙이 여러 파일에 흩어져 있었다

스킬 슬롯은 4칸, 마지막 슬롯은 3, 빈 칸은 -1. 이 세 값은 서로 종속인데 한 군데에도 그렇게 적혀 있지 않았다.

v0.3.3 — 같은 규칙이 최소 3개 계층에 리터럴로 반복됐다

```
// SaveData.cs - 저장
public int[] skillSlots = { 0, 1, 2, -1 };
for (int i = 0; i < Math.Min(4, skillSlots.Length); i++) n[i] = skillSlots[i];
if (skill < 0 || skill >= 3 || seenSkills[skill]) skillSlots[i] = -1;

// Game.cs - 게임 로직
S.skillSlots[slot] = -1;
while (slot < S.skillSlots.Length && S.skillSlots[slot] >= 0) slot++;

// GameUI.Mobile.cs - 표현 (같은 파일에서 8곳)
for (int slot = 0; slot < 4; slot++) { ... }
for (int s = 0; s < 4; s++) if (G.SkillAtSlot(s) == i) { slot = s; break; }
```

슬롯을 5칸으로 늘리는 작업을 상상해 보면 위험이 분명해진다. 4는 검색으로 찾을 수 있지만 같은 4가 심연 트리 티어 수에도, 퀵 미션 배열에도 있다. 전부 바꾸면 다른 시스템이 깨지고, 골라서 바꾸면 빠뜨린다.

0.4 — Assets/Scripts/Core/GameSystemRules.cs

```
/// Structural rules shared by simulation, save compatibility and presentation.
/// These are engine contracts, not tunable content values.
public static class GameSystemRules
{
    public static class SkillLoadout
    {
        public const int SlotCount = 4;
        public const int NoSlot = -1;
        public const int EmptyLegacyOrdinal = -1;
        public const int FallbackSlot = SlotCount - 1; // 종속 관계가 코드에 드러난다
        public static bool IsValidSlot(int slot) { return slot >= 0 && slot < SlotCount; }
    }

    public static class AbyssTree { public const int BranchCount = 3, TierCount = 4;
        public const int LastTier = FirstTier + TierCount - 1; }

    public static class ArenaPool { public const int ShotCapacity = 64; // 시뮬레이션과 렌더러가
        public const int HazardCapacity = 24; } // 같은 값을 본다

    /// Fixed v0.3 array layout retained only for rollback-compatible saves.
    public static class LegacyAbyssSave { public const int NodeSlotCount = 16; /* ... */ }
}
```

여기 들어가지 않은 것 — 밸런스 수치와 연출 값은 Balance와 콘텐츠 데이터에 남았다. GameSystemRules는 "바꾸면 여러 계층이 동시에 어긋나는 구조 규칙"만 받는다. 이 경계를 넓히면 상수 창고가 되고, 그건 원래 문제보다 나쁘다.

7 전역 접근이 테스트와 교체를 막았다

SaveIO 는 static 이었고 PlayerPrefs 를 직접 호출했다. 저장 경로를 바꿔 끼울 방법이 존재하지 않았다.

V0.3.3

```
public static class SaveIO
{
    const string Key = "shimcheung.save.v1";

    public static SaveData Load()
    {
        string json = PlayerPrefs.GetString(Key, "");
        ...
    }
    public static void Save(SaveData d) { ... }
}

public class Game : MonoBehaviour
{
    public static Game I;          // 전역 싱글턴
}

// Bootstrap.cs
Sfx.Ensure();
Bgm.Ensure();
root.AddComponent<Game>();
var ui = root.AddComponent<GameUI>();
ui.Build();                      // 인자 없음 =
Game.I를 찾아 씀
```

0.4

```
public interface IGameSaveStore
{
    SaveData Load(IGameContentCatalog content);
    void Save(SaveData data);
    void Wipe();
}

public interface IGameClock { long UnixNow { get; } }

public sealed class GameRuntimeServices
{
    public readonly IGameSaveStore Saves;
    public readonly IGameClock Clock;
    public readonly IGameContentCatalog Content;
}

// Bootstrap.cs - 조립 지점이 수명을 소유한다
var game = root.AddComponent<Game>();
game.Initialize(GameRuntimeServices.CreateLocal());
var sfx = Sfx.Create(() => game.S != null &&
game.S.muted);
var bgm = Bgm.Create(..., game.Services.Content.Zones);
var ui = root.AddComponent<GameUI>();
ui.Build(game, sfx, bgm);      // 명시적 전달
```

이 교체가 실제로 열어준 것

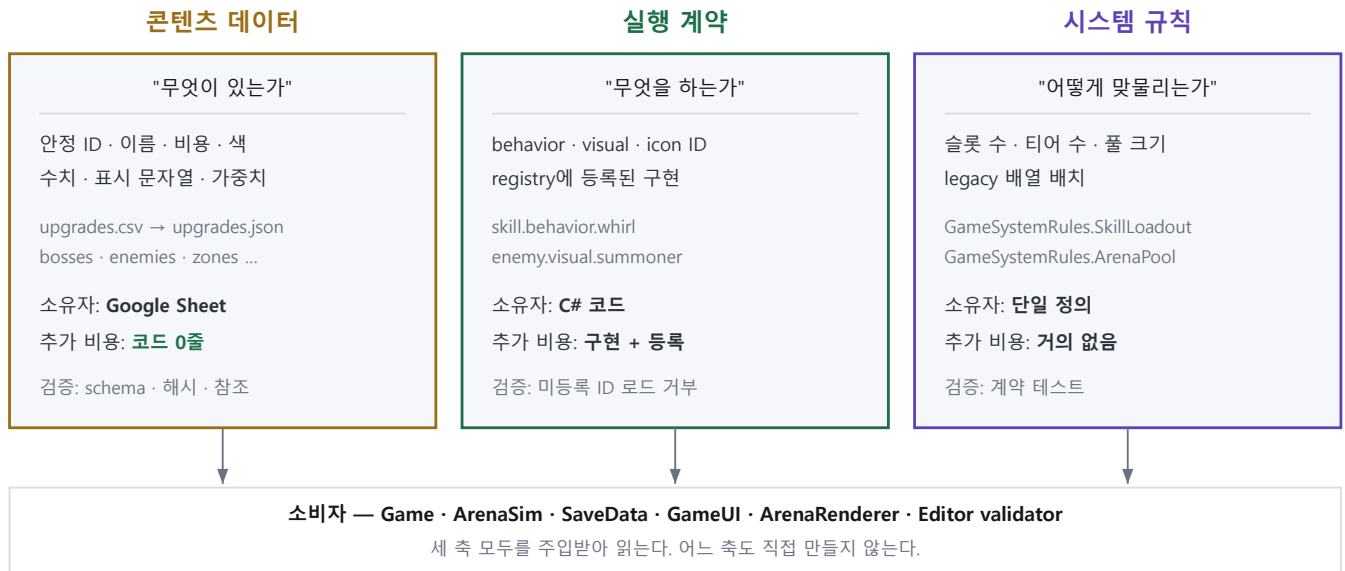
교체 대상	0.3.3	0.4
저장소	PlayerPrefs 고정	IGameSaveStore — 서버 저장소 · 테스트 대역으로 교체 가능
시계	DateTime.UtcNow 직접 호출	IGameClock — 오프라인 보상을 결정론적으로 테스트
콘텐츠	정적 배열	IGameContentCatalog — 내장 · 원격 · 메모리 스냅샷

13장의 validator 중 주입 저장소 · 시계 · 카탈로그 테스트와 10종 실패 차단 테스트는 이 교체 없이는 아예 작성할 수 없다. 메모리 스냅샷을 카탈로그로 주입할 수 있어야 "망가진 콘텐츠"를 만들어 볼 수 있기 때문이다.

과하게 하지 않은 부분 — DI 컨테이너를 도입하지 않았다. 조립 지점은 Bootstrap.Launch() 하나이고, 생성 순서가 코드에 그대로 보인다.

8 콘텐츠는 데이터, 실행은 계약, 구조는 시스템 규칙

네 위험은 전부 서로 다른 성질의 값이 한 곳에 섞여 있어서 생겼다. 해결은 새 기술이 아니라 **세 종류를 갈라 각자의 소유자에게 주는 것이었다.**

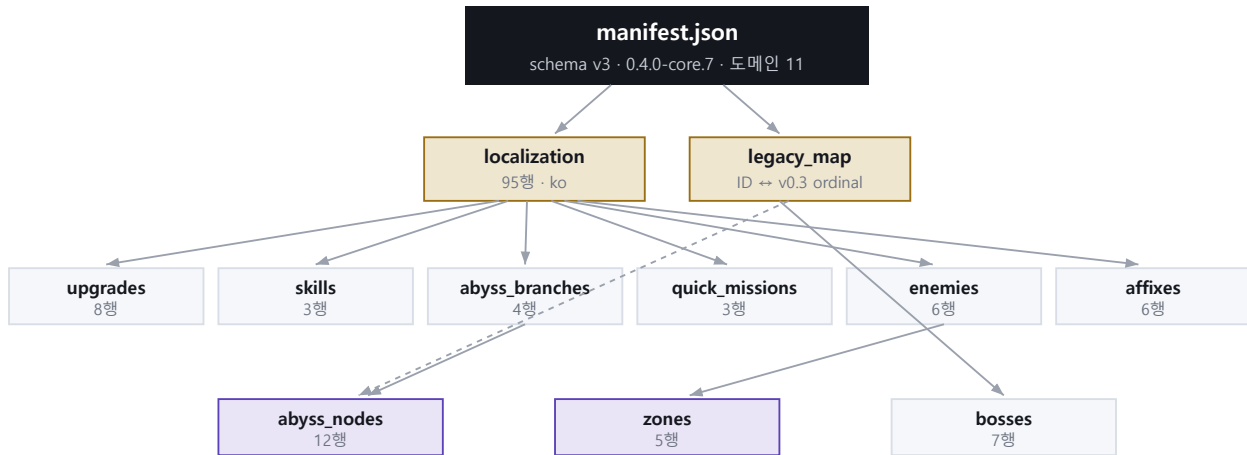


도식 1 — 값의 성질에 따라 소유자와 추가 비용이 달라진다. 판단 기준은 "이 값을 바꿀 때 무엇이 같이 바뀌어야 하는가"다.

이 3분할이 위험 4가지에 각각 대응한다. 위험 1 · 2는 **콘텐츠 데이터와 실행 계약의 분리**로, 위험 3은 **시스템 규칙의 단일 정의**로, 위험 4는 **세 축을 모두 주입으로 받는 소비자**로 풀린다.

9 콘텐츠 스냅샷 구조

단일 파일이 아니라 작은 manifest가 11개 도메인 파일을 가리키는 구조다. 스냅샷은 **물리적 파일이 아니라 논리적 버전 단위**다.



실선 = 선언된 의존성 · 점선 = legacy_map 참조. 의존 도메인이 없으면 로드 자체가 실패한다.

도식 2 — manifest → 11개 도메인 의존성 그래프. 행 수는 이 저장소의 0.4.0-core.7 실측값이다.

manifest.json — 도메인 엔트리 하나

```

{
  "id": "zones",
  "path": "Content/zones",
  "contentVersion": "0.4.0-core.7",
  "sha256": "ba4483b3fc8bfb80a20315ee506ee286c93d83d4898bc264508736d791bcb9a0",
  "schemaVersion": 1,
  "dependencies": [ "localization", "enemies" ]
}

```

도메인마다 자기 버전 · 해시 · 의존성을 들고 있으므로, 규모가 커지면 **필요한 팩만 내려받아 캐시**하고 모든 필수 팩 검증이 끝난 뒤 **호환되는 조합 전체를 원자적으로 활성화**하는 방향으로 확장된다. 0.4는 그 구조만 세웠고 실제 원격 다운로드를 구현하지 않았다 (15장).

10 무결성 계약

데이터를 코드 밖으로 꺼내면 **컴파일러의 보호가 사라진다**. 그 자리를 대신할 것을 같이 만들지 않으면 리팩터링이 아니라 **보호막을 벗긴 것**이 된다.

네 겹

겹	무엇을 막나	구현
schema 버전	클라이언트가 모르는 형식의 스냅샷	manifest <code>schemaVersion</code> · 도메인별 <code>schemaVersion</code>
콘텐츠 버전	도메인 일부만 갈아끼운 혼합 스냅샷	모든 도메인이 동일 <code>contentVersion</code> 이어야 통과
SHA-256	전송 · 편집 · 캐시 중 훼손	도메인 파일 텍스트의 해시를 manifest가 보관
의존성	참조 대상이 빠진 부분 배포	도메인 엔트리의 <code>dependencies</code> 선언 + 교차 참조 검사

Assets/Scripts/Core/ContentCatalog.cs — 로드 시 검사

```
string actualHash = ContentSnapshotIntegrity.Sha256(source.LoadText(path));
if (!string.Equals(actualHash, row.sha256, StringComparison.OrdinalIgnoreCase))
    throw new InvalidOperationException("Content domain checksum mismatch: " + id);
```

해시를 텍스트에 걸었더니 줄바꿈이 계약이 됐다

이 설계에는 배포 시점에만 드러나는 함정이 있었다. 해시는 **파일 텍스트** 기준인데, Windows에서 `core.autocrlf=true` 로 커밋하면 다른 사람이 클론할 때 Git이 줄바꿈을 **CRLF로 바꿔서 체크아웃**한다. 그러면 바이트가 달라지고, **카탈로그가 자기 스냅샷을 거부한다**. 게임이 시작조차 하지 못한다.

.gitattributes — 줄바꿈을 계약으로 고정

```
# Content snapshot – line endings are part of the contract.
# manifest.json stores a SHA-256 of each domain file's text, so a CRLF conversion on
# checkout would change the bytes and make the catalog reject its own snapshot at load.
Assets/Resources/Content/*.json      text eol=lf
ContentSource/GoogleSheets/*.csv     text eol=lf
```

추가로 `ContentSnapshotIntegrity.Sha256` 은 해싱 전에 **끝의 개행만 잘라낸다** (`TrimEnd('\r', '\n')`). 파일 끝 개행 한 칸 차이로 배포가 깨지지 않게 하는 안전판이다.

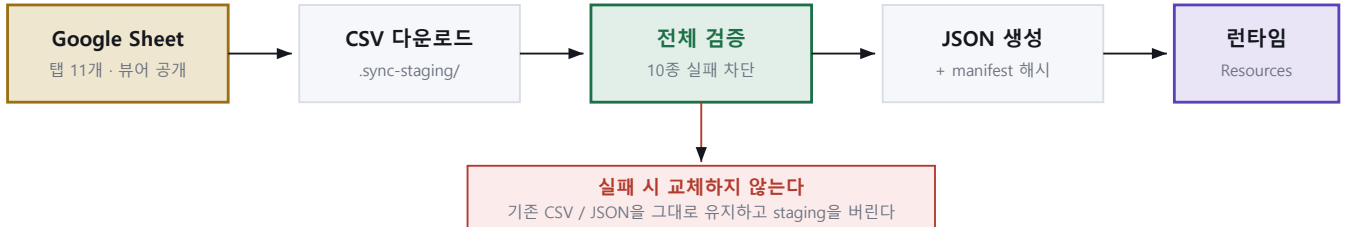
독립 재계산 결과

Unity 밖에서 저장소 파일을 직접 읽어 SHA-256을 다시 계산했다 (2026-08-12).

대상	일치	비고
워킹 트리 파일 11개	11 / 11	디스크 바이트 그대로
Git 인덱스 blob 11개	11 / 11	LF 정규화된 커밋 내용
도메인 <code>contentVersion</code>	11 / 11	전부 <code>0.4.0-core.7</code>

11 제작 파이프라인

밸런스를 고치는 사람이 **Unity도 Git도 열지 않아도** 되게 만든다. 대신 잘못된 데이터는 **런타임이 아니라 동기화 단계에서** 막는다.



진입점 — Tools / Sync Google Sheet Content (Ctrl+Shift+G) · 최초 연결은 Tools / Abyss / Content / Connect Published Google Sheet...
비공개 시트는 탭별 UTF-8 CSV 수동 경로를 쓴다. 두 경로 모두 같은 importer와 같은 검증을 통과한다.

도식 3 — Sheets → CSV → 검증 → JSON → 런타임. 검증이 통과하지 못하면 저장소의 기존 데이터는 한 글자도 바뀌지 않는다.

한 행이 화면까지 가는 길

ContentSource/GoogleSheets/upgrades.csv

```

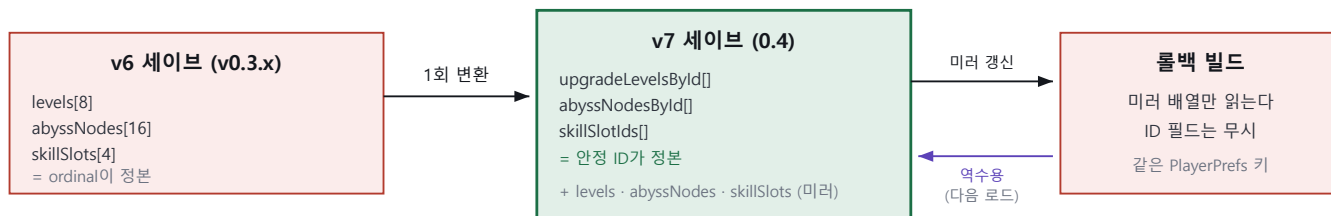
id,effect_id,name_key,icon_id,base_cost,cost_multiplier,ui_accent_color
upgrade.damage,upgrade.effect.damage,upgrade.damage.name,icon.upgrade.sword,10,1.09,#FF6157
upgrade.speed,upgrade.effect.attack_speed,upgrade.speed.name,icon.upgrade.speed,32,1.125,#59D9FF
upgrade.crit_chance,upgrade.effect.crit_chance,upgrade.crit_chance.name,icon.upgrade.crit,120,1.16,#FFD44D
  
```

강화	심연	MAX
검날 연마 Lv.138 공격력 54.3K		1.46M G +1 레벨
속공 수련 Lv.106 초당 7.03회		8.46M G +1 레벨
급소 간파 Lv.43 치명타 확률 39.6%		70.9K G +1 레벨
처형 각인 Lv.48 치명타 피해 x8.24		120K G +1 레벨
탐욕의 손 Lv.23 골드 획득 x3.07		1.74K G +1 레벨
그림자 잔영 Lv.27 자동 피해 276/초		17.3K G +1 레벨
강철 체력 Lv.15 최대 체력 x2.2 · 재생 2.8%/초		1.00K G +1 레벨
질풍 보법 Lv.14 이동 속도 +49%		718 G +1 레벨

강화 8행의 이름 · 아이콘 · 색 · 비용 곡선이 위 CSV에서 온다. effect_id 만 코드가 소유하는 실행 계약이고, 나머지 6개 컬럼은 편집자의 것이다.

12 세이브 호환 — v6 → v7

이미 플레이 중인 세이브가 있다. 그리고 **구버전 빌드로 롤백할 가능성도 있다**. 두 방향을 모두 살려야 **배 포된 게임의 리팩터링**이 된다.



세 가지 규칙

- ① 정본은 안정 ID다. 미러는 **저장 직전에 정본에서 다시 생성한다**.
- ② 구버전으로 플레이해 미러가 달라졌으면, 다음 로드에서 **알려진 ID에 한해 역수용**한다.
- ③ 구버전이 모르는 **미래 ID는 삭제하지 않고 보존**한다 — 서버/신규 클라이언트 데이터를 구버전 캐시가 훼손하지 않게.

도식 4 — v6 → v7 변환과 롤백 미러의 양방향. PlayerPrefs 키와 JSON 외형은 v0.3.3과 동일하다.

SaveData.cs — 미러를 정본으로 되돌리는 지점

```

public void SetLv(string stableId, int v)
{
    SetStableValue(ref upgradeLevelsById, Content.Upgrade(stableId).StableId, Math.Max(0, v));
    int ordinal = Content.Legacy.UpgradeOrdinal(stableId); // legacy_map.json
    if (ordinal >= 0 && ordinal < levels.Length) levels[ordinal] = Math.Max(0, v);
}

// EnsureShape() 안 - v7로 올린 뒤에도 구버전이 만진 미러를 확인한다
if (version < 7) { ... version = 7; }
if (loadedVersion >= 7) ReconcileRollbackMirrors();
SyncLegacyProgression();
  
```

이 규칙이 잡아낸 실제 결함 — 13장의 **기본 스킬 슬롯 계약 테스트**가, legacy 슬롯이 없는 **새 v7 세이브**에서 롤백 조정 코드가 기본 스킬을 비워버리는 버그를 발견했다. 실제 legacy 슬롯이 있을 때만 조정하도록 고쳤다. 테스트가 없었다면 신규 플레이 어만 스킬 없이 시작했을 것이다.

13 영구 검증

한 번 통과시키고 끝나는 검증은 결과가 아니라 기분이다. 모든 **Player 빌드 전에** 자동으로 돌고, 실패하면 빌드가 중단된다.

Assets/Editor/GameContentBuildValidator.cs

```
[MenuItem("Tools/Abyss/Validate Game Content")] public static void ValidateFromMenu() { ... }

// IPreprocessBuildWithReport - 모든 Player 빌드 앞에 자동으로 붙는다
catch (Exception e) { throw new BuildFailedException("Game content validation failed: " + e.Message); }
```

① 콘텐츠 개수 계약 — 리팩터링이 콘텐츠를 잃지 않았다는 가장 단순한 증거

```
Require(catalog.Upgrades.Length == 8, "Expected 8 v0.3.3 upgrades");
Require(catalog.Skills.Length == 3, "Expected 3 v0.3.3 skills");
Require(catalog.AbyssBranches.Length == 4, "Expected 4 abyss branch definitions");
/* quick_missions 3 · nodes 12 · archetypes 6 · affixes 6 · zones 5 · bosses 7 도 같은 방식 */
```

② 실패 차단 10종 — 망가진 스냅샷을 만들어 거부되는지 본다

메모리 스냅샷을 주입할 수 있게 만든 7장의 경계가 여기서 값을 한다. **고의로 훼손한 콘텐츠가 특정 예외로 거부되는지** 확인한다. 전체 목록은 부록 C.

③ 계약 스위트 5종

스위트	확인하는 것
세이브 호환	v6 → v7 변환값, JSON 왕복 동일성, 미래 변경의 역수용
미래 ID 보존	알 수 없는 ID가 로드 · 저장을 거쳐도 사라지지 않는다
결정론적 층 구성	같은 층 · 같은 슬롯이면 항상 같은 적 구성이 나온다
기본 스킬 슬롯	새 세이브가 기본 스킬 3종을 들고 시작한다 — 12장의 결함을 잡은 테스트
주입 런타임 서비스	저장소 · 시계 · 카탈로그를 대역으로 주입해 Game 이 동작한다

실행 결과 — 2026-08-12

항목	결과
Unity 6000.5.1f1 전체 컴파일	오류 0 · 경고는 NodeDef.Id 사용처 4곳의 CS0618 뿐 (의도적, 부록 B)
Tools/Abyss/Validate Game Content	통과 — 0.4.0-core.7
manifest 무결성 · 시트 대조	11 / 11 도메인 체크섬 · 버전 일치 (저장소 밖 독립 재계산, 10장) · 시트 11개 탭 중 10개 바이트 일치, zones 만 55.00 → 55
Play Mode 콘솔 · Windows64 빌드	오류 0 / 경고 0 · 런타임 개수 8/3/4/3/12/6/6/5/7 · 272.83MB 빌드 오류 0 · PlayerPrefs는 테스트 전후 SHA-256 C8D5B6C5...4C130 로 정확히 복원

14 결과 — 코드 변경 표면

이 리팩터링은 LOC를 줄이지 않았다. 총량은 늘었고, 늘어난 쪽은 검증 · 호환 · 도구 코드다. 그러니 **"무엇이 줄었나"가 아니라 "무엇이 바뀌기 쉬워졌나"**로 읽는다.

지표	V0.3.3	0.4	해석
UpKind · SkillKind · IconId · BossKind 직접 참조	228	0	콘텐츠 종류가 코드 식별자에서 완전히 분리됐다
Balance.cs + Balance.Arena.cs	1,020	712 (-308)	정적 정의표가 데이터로 이동했다
Assets/Scripts 전체	13,121	14,541 (+1,420)	검증 · 호환 · 서비스 경계가 늘어난 몫
Editor 도구	258	1,502 (+1,244)	importer · validator · 시트 연결 도구
콘텐츠 행 1개를 추가할 때 고쳐야 하는 코드	enum + 배열 + switch	0줄	행동을 재사용하는 한 데이터만 편집한다

측정 기준 — 기준 커밋 0386937 (v0.3.3)의 트리를 git archive 로 추출해 현재 워킹 트리와 같은 방식으로 계산했다. .cs 파일의 물리 줄 수이고, 주석 · 빈 줄을 포함한다. 측정 스크립트는 부록 A에 있다.

보스를 하나 추가한다면

단계	V0.3.3	0.4
1	enum BossKind 에 항목 추가	시트 bosses 탭에 행 하나 추가
2	BossCombatNames[] 에 이름 추가	localization_ko 에 이름 한 줄
3	BossTint() switch에 case 추가	
4	BossVisualHeight() switch에 case 추가	Ctrl+Shift+G 동기화 → 검증 통과 → 끝 기존 패턴 재사용 시 코드 변경 0
5	BossHasSecondPhase() 조건에 추가	
6	렌더러 프레임 선택 분기 추가 · 재컴파일 · 재배포	새 패턴이 필요할 때만 behavior registry에 구현 등록

정직하게 말하면 — 이 구조의 값은 0.5부터 회수된다. 0.4 시점에서 콘텐츠는 여전히 8 · 3 · 4 · 3 · 12 · 6 · 6 · 5 · 7행이고, 늘어난 1,244줄의 Editor 코드는 아직 아무것도 되돌려주지 않았다. 이 투자를 정당화하는 것은 현재 규모가 아니라 **장비 6슬롯 · 4등급 · 옵션 풀이 들어오는 다음 버전**이다.

15 남은 위험과 0.5 계획

실행하지 않은 검증은 근거로 쓰지 않는다. 이 장은 **0.4가 확인하지 못한 것**을 숨기지 않고 적는 자리다.

미검증 — 다음 담당자가 확인할 것

항목	상태
GoogleSheetConnectWindow 의 gid 자동 검색	한 번도 성공 실행된 적이 없다. 현재 연결 설정은 브라우저로 탭을 열어 손으로 적은 gid다
Windows / WebGL 빌드	0.4.0-core.6 시점 산출물이다. 스냅샷이 Resources 로 패키징되므로 core.7 은 재빌드해야 반영된다
Android / iOS 실기기	이 구간에서 확인하지 않았다
원격 콘텐츠 다운로드 · 캐시 · 서명	미구현. 서버는 도입하지 않았고 현재 단계에 필요하지 않다고 판단했다

이번에 새로 확인한 함정

- Unity 메인 스레드에서 **async**를 블로킹하면 에디터가 죽는다. `GetAwaiter().GetResult()` 로 동기 래퍼를 만들자 continuation이 Unity SynchronizationContext로 돌아오는데 그 스레드를 이미 막고 있어 영구 데드락이 낫다. `Task.Run(...)` + `ConfigureAwait(false)` 로 고쳤다.
- `EditorUtility.DisplayDialog` 는 메인 스레드를 잡는다. `finally` 정리보다 **뒤에** 두어야 staging 폴더가 제때 지워진다.



0.5 — 파밍 루프와 D2식 아이템

왼쪽 화면 하단의 장비 6슬롯은 **0.5의 비영속 미리보기**다. 저장되지 않고 레이아웃 예산만 확인한다.

0.5는 층 고정을 파밍 모드로 개편하고, 존을 티어로 삼는 장비 드롭을 넣는다. 티어당 예산 배율 $k = 2.0$ 은 $k^{0.05} < 1.169$ 를 만족해야 기존 성장 벽이 유지된다.

0.5 원칙 6 — 세이브에는 파생 수치를 굳히지 않는다. 안정 ID · 티어 · 롤 시드처럼 **재계산의 입력만** 저장하고, 최종 전투력은 로드할 때마다 다시 만든다. 이걸 어기는 순간 세이브가 콘텐츠 스냅샷의 사본이 되어 **0.4가 만든 경계가 무너진다**.

착수 조건은 코드가 아니라 **플레이 판정**이다 — "몸이 달라도 똑같이 느껴진다"가 나오면 D2식 드롭은 등급 색만 화려한 무의미가 된다. 구조는 준비됐고, 다음 결정은 게임 쪽에 있다.

A 콘텐츠 스키마와 측정 방법

11개 도메인 컬럼 계약

도메인	행	컬럼
localization_ko	95	key, value
legacy_map	—	JSON 전용. upgradeIds, skillIds, nodeIds, archetypeIds, affixIds, bossIds
upgrades	8	id, effect_id, name_key, icon_id, base_cost, cost_multiplier, ui_accent_color
skills	3	id, behavior_id, name_key, description_key, icon_id, duration_seconds, cooldown_seconds, damage_multiplier, attack_speed_multiplier, reach_multiplier, range, projectile_speed, ui_accent_color (13)
abyss_branches	4	id, name_key, role_key, display_order, ui_accent_color, selectable
quick_missions	3	id, trigger_id, name_key, short_name_key, target, ruby_reward
abyss_nodes	12	id, effect_id, branch_id, tier, name_key, glyph, base_cost, cost_multiplier, max_level
enemies	6	id, behavior_id, visual_id, name_key, speed, reach, keep_distance, attack_interval_seconds, damage_multiplier, health_multiplier, body_radius, ui_accent_color + 아키타입별 aura_* · blast_radius · dies_on_attack · summon_interval_seconds · split_* · ranged (20)
affixes	6	id, behavior_id, name_key, note_key, ui_accent_color, health_multiplier, damage_multiplier, speed_multiplier
zones	5	id, name_key, background_top_color, background_bottom_color, ui_accent_color, music_root_hz, enemy_name_keys, archetype_weights
bosses	7	id, behavior_id, visual_id, name_key, ui_accent_color, visual_height, has_second_phase

엄격한 입력 계약 — CSV는 누락 · 중복 · 추가 컬럼과 헤더보다 긴 행을 모두 거부한다. 관대한 파서는 오타를 삼켜 런타임 기 본값으로 나타낸다.

14장 수치의 측정 방법

```
# v0.3.3 트리를 꺼내 현재 워킹 트리와 같은 방식으로 센다
git archive 0386937 Assets/Scripts Assets/Editor -o base033.zip

# .cs 파일의 물리 줄 수 합계 (주석·빈 줄 포함)
#   Assets/Scripts 13,121 → 14,541   Assets/Editor 258 → 1,502
#   Balance.cs + Balance.Arena.cs 1,020 → 712

# enum 직접 참조 - 단어 경계 정규식으로 .cs 전체 검색
#   \bUpKind\b 33→0   \bSkillKind\b 80→0   \bIconId\b 43→0   \bBossKind\b 72→0   (228 → 0)
# manifest 무결성 - Unity 밖에서 도메인 파일 텍스트를 TrimEnd('\r','\n') 후 SHA-256 (10장)
```

B legacy_map.json 과 ordinal 격리 지점

ordinal은 **없앤 것이 아니라 가둔 것이다**. v0.3.x 세이브가 존재하는 한 사라질 수 없고, 대신 **네 곳에서만** 존재한다.

Assets/Resources/Content/legacy_map.json (전문)

```
{
  "upgradeIds": [
    "upgrade.damage", "upgrade.speed", "upgrade.crit_chance", "upgrade.crit_damage",
    "upgrade.gold", "upgrade.companion", "upgrade.vitality", "upgrade.move"
  ],
  "skillIds": ["skill.frenzy", "skill.whirl", "skill.throw"],
  "nodeIds": [
    "abyss.exec.t1", "abyss.exec.t2", "abyss.exec.t3", "abyss.exec.t4",
    "abyss.plunder.t1", "abyss.plunder.t2", "abyss.plunder.t3", "abyss.plunder.t4",
    "abyss.echo.t1", "abyss.echo.t2", "abyss.echo.t3", "abyss.echo.t4",
    null, null, null, null
  ],
  "archetypeIds": [ "enemy.rusher", "enemy.archer", "enemy.warder",
    "enemy.bomber", "enemy.splitter", "enemy.summoner" ],
  "affixIds": [ "affix.fire", "affix.cold", "affix.lightning",
    "affix.magic_resist", "affix.berserker", "affix.regen" ],
  "bossIds": [ "boss.mine_lord", "boss.crystal_queen", "boss.furnace_tyrant",
    "boss.high_priest", "boss.sunken_king", "boss.rift_devourer",
    "boss.sky_calamity_dragon" ]
}
```

nodeIds의 마지막 null 4칸이 이 파일의 성격을 보여준다. v0.3의 심연 배열은 16칸이었고 그중 12칸만 살아 있는 노드다. 나머지 4칸은 **삭제된 각인 T5 · 선불 노드의 자리**이며, 환급 로직이 그 위치를 알아야 하므로 비워둔 채 남긴다.

ordinal이 존재하는 네 곳

#	위치	역할
1	Content/legacy_map.json	안정 ID ↔ v0.3 ordinal 대응표. 유일한 정보이며 데이터로 존재한다
2	GameSystemRules.LegacyAbyssSave	v0.3 심연 배열의 고정 배치 — 슬롯 16칸, 각인 T5 3칸, 선불 1칸. 환급 코드 전용
3	SaveData.SyncLegacyProgression() · SyncLegacySkillSlots()	저장 직전 미리 재생성. ordinal을 쓰기만 하고 읽어서 판단하지 않는다
4	[Obsolete] NodeDef.Id · Node(int index) 오버로드	v0.3 세이브 경계 전용 접근자. 컴파일 경고 CS0618 4건이 여기서 나온다

경고를 끄지 않은 이유 — CS0618 4건은 억제하지 않고 남겼다. 이 경고는 "여기가 legacy 경계다"라는 **표시**로 동작한다. v0.3.x 롤백 지원을 끝내는 날, 경고가 사라지는 지점이 곧 삭제할 코드의 목록이 된다.

C validator 실패 차단 10종

각 항목은 고의로 훼손한 스냅샷을 메모리에 만들어 카탈로그가 특정 예외로 거부하는지 확인한다. 하나라도 통과하면 빌드가 실패한다.

#	훼손 방식	기대 예외
1	manifest schemaVersion 을 999로	Unsupported content schema
2	upgrades 본문에 공백 한 칸 추가	checksum mismatch
3	강화 2행의 id 를 1행과 같게	Duplicate upgrade ID
4	nameKey 를 존재하지 않는 키로	Unknown localization key
5	스킬 behaviorId 를 미등록 ID로	No registered skill behavior
6	적 visualId 를 미등록 ID로	No registered enemy visual
7	zones 가 참조하는 적 행을 enemies 에서 실제로 삭제	Unknown archetype
8	노드의 branchId 를 없는 분기로	Unknown abyss branch
9	bosses 도메인만 다른 contentVersion 으로	Mixed content versions
10	zones 의 dependencies 에 없는 도메인 선언	Missing content domain dependency

7번이 삭제인 이유 — 처음에는 존이 없는 ID를 참조하게 만들었는데, 그러면 legacy_map 계층이 먼저 걸러내서 크로스 도메인 검증까지 도달하지 않았다. 테스트가 통과했지만 의도한 코드를 지나지 않은 것이다. 그래서 실제로 enemies.json 에서 행을 제거하는 방식으로 바꿨다. 테스트가 통과했다는 사실만으로는 그 테스트가 무엇을 지키는지 알 수 없다.

이 10종이 늘어난 경위

초기 6종(1-6)은 스냅샷 자체의 형식과 ID를 본다. 나머지 4종(7-10)은 DESIGN_0.4.md 완료 조건 4번의 "참조" 항목이 계약 문서에만 있고 테스트가 없다는 것을 마감 점검에서 발견해 추가했다. 계약과 테스트의 불일치는 계약이 지켜지지 않는 것과 같다.

D 편집자용 Google Sheets 운영 가이드

이 파이프라인의 목표는 밸런스를 만지는 사람이 Unity를 열지 않는 것이다. 아래가 편집자에게 필요한 전부다.

일상 흐름

1. Google Sheet에서 해당 탭의 값을 고친다. 탭 이름 = 도메인 이름이다.
2. 개발자가 Unity에서 `Tools / Sync Google Sheet Content` (`Ctrl+Shift+G`)를 누른다.
3. staging 다운로드 → 전체 검증 → CSV/JSON 교체가 한 번에 돈다.
4. 검증에 실패하면 아무것도 바뀌지 않는다. 오류 메시지가 어느 탭 · 어느 행인지 알려준다.

규칙

대상	규칙
컬럼	추가 · 삭제 · 이름 변경 금지. 헤더가 계약이다
<code>id</code>	한 번 배포되면 바꾸지 않는다. 세이브가 이 값을 가리킨다
<code>behavior_id</code> · <code>visual_id</code> · <code>icon_id</code> · <code>effect_id</code>	코드가 소유한다. 목록에 없는 값을 쓰면 동기화가 거부된다
<code>name_key</code> 등 <code>*_key</code>	<code>localization_ko</code> 탭에 같은 키가 있어야 한다
색	<code>#RRGGBB</code> 형식. 금색과 보라는 골드 · 심연석이 이미 점유했다
행 순서	자유롭게 바뀌도 된다. 세이브는 순서가 아니라 <code>id</code> 를 본다
<code>settings!B2</code>	콘텐츠 버전. 의미 있는 변경을 했으면 올린다

연결 설정

시트는 링크가 있는 모든 사용자 / 뷰어로 공개되어야 CSV export를 읽을 수 있다. 문서 ID와 탭별 `gid`는 로컬 `ContentSource/GoogleSheets/google_sheet_sync.json`에 들어가며, 이 파일은 Git에서 제외된다. 저장소에는 `google_sheet_sync.example.json`과 한국어 편집 가이드만 공유한다.

`Tools / Abyss / Content / Connect Published Google Sheet...`는 시트 주소를 받아 문서 ID를 뽑고 공개 `htmlview`에서 탭 이름 → `gid`를 찾아 연결 설정을 쓴다. 이 자동 검색은 아직 성공 실행이 확인되지 않았다 (15장). 실패해도 수동 입력 경로가 남는다.

비공개 시트

공개하지 않으려면 탭별로 UTF-8 CSV를 내려받아 `ContentSource/GoogleSheets/`에 덮어쓰고 CSV 빌드 메뉴를 실행한다. 검증은 동일하게 통과해야 한다 — 수동 경로라고 느슨해지지 않는다.

문서 기준 — 심층 0.4 · 콘텐츠 0.4.0-core.7 · 커밋 4ef06cb · Unity 6000.5.1f1 · 작성 2026-08-12. 관련 문서:

DESIGN_0.4.md (설계 계약), DESIGN_0.5.md (다음 버전), HANDOFF.md (인계), ContentSource/GoogleSheets/README.md (편집 가이드).